UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO
PRIMEIRA LISTA DE PROGRAMAÇÃO-FUNCIONAL – PARTE 1

Profa. Leila Maciel de Almeida e Silva Data de entrega: 17/12/2019 (T06)
18/12/2019 (T04)

Aula: Tipos Básicos e Definições

1. Dados números inteiros elabore:
   a. Uma função `tresDiferentes` que determina se três inteiros são diferentes.
   b. Uma função `quatroDiferentes` que determina se os quatro inteiros são diferentes, sem usar a função do item (a);
   c. Usando a função `tresDiferentes`, do item (a), refaça o item (b).
   d. Usando definições locais `let` e depois `where` refaça o item (b).
   e. Estabeleça os casos de teste para todas as funções pedidas.
   f. Execute as funções dos itens (a) a (d) no GHCi e teste-as para os casos de teste que estabeleceu. Anexe ao arquivo com os programas outro arquivo com as imagens das telas obtidas com a execução.
2. Dados três inteiros defina:
   a. Usando 1(a) e a função `tresIguais`, vista em sala de aula, elabore uma função para saber quantos números são iguais.
   b. Usando (a) elabore uma função para saber, dados quatro inteiros, quantos são diferentes.
   c. Estabeleça os casos de teste para todas as funções pedidas.
   d. Execute as funções dos itens (a) a (c) no GHCi e teste-as para os casos de teste que estabeleceu. Anexe ao arquivo com os programas outro arquivo com as imagens das telas obtidas com a execução.

Aula: Tuplas e Listas

As questões a seguir devem ser feitas usando **compreensões** e as **funções básicas sobre listas e tuplas.** Todas elas estão relacionadas ao cenário a seguir descrito.

3. Considere um sistema para gerenciar o cômputo de notas de uma competição de ginástica artística feminina. A competição armazena um cadastro das competidoras que é atualizado durante o período de inscrição das ginastas. Cada ginasta tem um nome, um número inteiro, que é o seu identificador de registro na competição, e está vinculada a um país, que é identificado pelas três letras iniciais do país, em maiúsculo.

```
type NomeGin = String
type IdGin = Int
type PaisGin = String
type Ginasta = (IdGin, NomeGin, PaisGin)
type Competidoras = [Ginasta]
```

Um exemplo de um cadastro de competidoras poderia ser:

```
cadastro :: Competidoras
cadastro = [(150, "Luiza Parente", "BRA"), (15, "Sue Smith", "USA"), (200, "Maria Pontes", "POR"), (204, "Julia Benfica", "POR")]
```

A competição envolve quatro aparelhos e em cada aparelho os juízes atribuem uma nota à ginasta. Existem vários juízes em um aparelho e o número de juízes pode variar por aparelho. As notas de cada ginasta são armazenadas em uma lista de listas de notas. Cada posição desta lista é representada por um aparelho. Assim a lista de notas da primeira posição refere-se ao solo, a da segunda posição às barras assimétricas, a da terceira posição ao cavalo, a da quarta posição à trave. É possível que uma nota digitada por um juiz seja alterada, por exemplo, para o atendimento a um recurso da atleta ou porque foi informada duplicada. O atleta pode incorrer em penalidades, representada por um número inteiro numa escala de 0 a 50, para cada aparelho. Cada juiz pode lançar uma penalidade juntamente com a nota. A nota final de cada juiz será a nota menos a penalidade atribuída por ele. O sistema deve computar as médias das atletas por aparelho e em todos os aparelhos e exibir todas as notas finais e a média para o público. As notas são números inteiros numa escala de 0 a 100, mas devem ser exibidas com uma casa decimal, numa escala de 0 a 10, e para isso uma formatação da nota e da média deve ser realizada antes da exibição ao público. A média final das notas do aparelho será a média das notas finais de todos os juízes daquele aparelho.

```
type IdAp = Int -- no intervalo de 1 a 4, conforme explicado
type NotaJuiz = Int
type Penalidade = Int
type Avaliacao = (NotaJuiz, Penalidade)
type MediaFinal = Int
type NotasGinastas = [(IdGin, [[Avaliacao]])]
```

O sistema funciona da seguinte maneira: para cada aparelho, os juízes atribuem notas e penalidades a cada atleta. Estas informações podem ser alteradas ou removidas. As notas, penalidades e a média de cada atleta são exibidas após a apresentação do atleta para o público. No final da competição do aparelho as médias dos atletas são exibidas em ordem decrescente. No final de todas as provas, a média geral é também exibida, em ordem decrescente.

Para desenvolver o sistema para esta situação problema pede-se:

3.1 Escreva as funções a seguir para manipular o cadastro de competidoras armazenado no sistema.

(a) Adiciona uma ginasta no cadastro. Se o identificador da ginasta já existir no cadastro deve retornar uma mensagem de erro sinalizando que existe um ginasta já cadastrado para aquele identificador.

```
adicionaGinasta :: Competidoras -> Ginasta -> Competidoras
```

(b) Remove uma ginasta no cadastro, informando seu identificador. Se o identificador da ginasta não existir no cadastro deve retornar uma mensagem de erro sinalizando que não existe uma ginasta no cadastro para aquele identificador.

```
removeGinasta :: Competidoras -> IdGin -> Competidoras
```

(c) Coleta dados de uma ginasta no cadastro, informando seu identificador. Para simplificar, considere que esta operação só tem o caso de sucesso, ou seja, o identificador consultado sempre vai existir no cadastro.

```
coletaGinasta :: Competidoras -> IdGin -> Ginasta
```

3.2 Escreva funções que permitam realizar ações que normalmente ocorrem na competição.

(a) Para uma dada ginasta, adiciona a avaliação de um juiz em uma lista de notas e penalidades de um dado aparelho. Esta função assume, por simplicidade, que o identificador da ginasta e do aparelho são valores válidos.

```
adicionaAval :: NotasGinastas -> IdGin -> IdAp -> Avaliacao ->
                                                    NotasGinastas
```

(b) Remove uma avaliação de um juiz em um dado aparelho. Por simplicidade, suponha que a avaliação a ser cancelada existe na lista de avaliações da ginasta e que o identificador da ginasta e do aparelho são valores válidos.

```
cancelaAval :: NotasGinastas -> IdGin -> IdAp -> Avaliacao ->
                                                NotasGinastas
```

(c) Gera uma tupla contendo: país, nome da ginasta, lista de notas, lista de penalidades e média final. Estas informações serão exibidas para o público após a apresentação da ginasta em cada aparelho. Para simplificar a questão, você pode supor que o IdGin é sempre válido, e que para toda atleta há uma lista de avaliações não vazia.

```
infoGinAp :: Competidoras -> NotasGinastas -> IdGin -> IdAp ->
      (PaisGin, IdGin, NomeGin, [NotaJuiz], [Penalidade],  MediaFinal)
```

(d) Gera a informação do placar de um dado aparelho ao final da competição do aparelho, ainda sem formatação. Suponha, por simplicidade, que as ginastas não possuem uma lista de avaliação vazia.

```
placarAparelho :: Competidoras -> NotasGinastas -> IdAp ->
[(PaisGin, IdGin, NomeGin, MediaFinal)]
```

(d) Gera a informação do placar global, ainda sem formatação. A média geral é a soma das médias finais de cada aparelho. Suponha, por simplicidade, que as ginastas não possuem uma lista de avaliação vazia, em nenhum dos aparelhos.

```
type MediaGeral = Int
placarGeral :: Competidoras -> NotasGinastas ->
            [(PaisGin, IdGin, NomeGin, [MediaFinal], MediaGeral)]
```

(d) Ordena as médias das atletas para exibição no placar por aparelho e geral. As atletas são ordenadas em ordem decrescente das médias.

```
ordenaGin :: [(IdGin, Int)] -> [(IdGin, Int)]
```

3.3 Escreva funções para formatar as notas e médias.

(a) Formata uma nota, média ou penalidade em uma string com uma casa decimal, justificada à direita. O total previsto para cada valor são 6 caracteres. Caso o valor não ocupe 6 caracteres você deve preencher com '.'.

```
formataValor :: Int -> String
```

`formataValor 2` retorna "`...0.2`"
`formataValor 99` retorna "`...9.9`"
`formataValor 205` retorna "`..20.5`"

(b) Formata e exibe as informações de uma ginasta após a apresentação de cada aparelho. O país deve usar 3 caracteres, o nome da ginasta 20 caracteres e o identificador 4 caracteres. Cada nota, média ou penalidade 6 caracteres, justificados à direita. O nome do país e da ginasta deve vir justificado à esquerda. As notas, média e penalidades devem ser formatadas usando a função do item 3.3.(a). O display final deve obedecer ao seguinte formato e para isso, `\n` deve ser usado após o nome da ginasta, da lista de notas e da lista de penalidades. Para exibir a string depois de formatada você deve usar `putStr`. Você deve usar a função `infoGinAp` como função auxiliar.

```
exibeInfoGin :: Competidoras -> NotasGinastas -> IdGin -> IdAp -> IO ()
```

Suponha que após o uso da função `infoGinAp` você obtenha a tupla
`("BRA","150", "Luiza Parente",[100,91,92],[8,0,2],91)`

Esta função deve retornar

```
BRA Luiza Parente (ID 150)
NOTAS ..10.0...9.1...9.2
PENAL ...0.8...0.0...0.2
MEDIA ...9.1
```

(c) Formata e exibe a informação do placar parcial por aparelho. A exibição obedece à ordem decrescente de médias. A saída da função deve obedecer ao formato do exemplo abaixo e usar a mesma quantidade de caracteres estipulados no item (b). Os ids das atletas devem ser preenchidos com 0 à esquerda, se necessário, para completar 4 caracteres. Você deve usar as funções `placarAparelho` e `ordenaGin` e outras funções auxiliares para organizar a informação necessária.

```
infoPlacarAp :: Competidoras -> NotasGinastas -> IdAp ->-> IO ()
```

Suponha que após o uso das funções auxiliares você obtenha a lista para o aparelho cavalo.

```
[("POR", 200, "Maria Pontes",95), ("BRA", 150, "Luiza Parente",
91), ("USA", 15, "Sue Smith", 90), ("POR", 204, "Julia
Benfica",88)]
```

Esta função deve retornar

```
CAVALO

POR 0200 Maria Pontes...........9.5
BRA 0150 Luiza Parente..........9.1
USA 0015 Sue Smith..............9.0
POR 0204 Julia Benfica..........8.8
```

(d) Formata e exibe a informação do placar geral. A exibição obedece à ordem decrescente de médias. A saída da função deve obedecer ao formato do exemplo abaixo e usar a mesma quantidade de caracteres estipulados nos itens anteriores. Você deve usar as funções `placarGeral` e `ordenaGin` e outras funções auxiliares para organizar a informação necessária.

```
infoPlacarGer :: Competidoras -> NotasGinastas -> IO ()
```

Suponha que após o uso das funções auxiliares você obtenha a lista para o placar geral.

```
[("POR", 200, "Maria Pontes",[95, 96, 95, 94], 380), ("BRA", 150,
"Luiza Parente", [90, 91, 91, 92], 364), ("USA", 15, "Sue Smith",
[90,90,90,90] 360), ("POR", 204, "Julia Benfica",[87,89,88,88],
352)]
```

Esta função deve retornar

```
PLACAR GERAL

POR 0200 Maria Pontes...........9.5...9.6...9.5...9.4..38.0
BRA 0150 Luiza Parente..........9.0...9.1...9.1...9.2..36.4
USA 0015 Sue Smith..............9.0...9.0...9.0...9.0..36.0
POR 0204 Julia Benfica..........8.7...8.9...8.8...8.8..35.2
```

Para testar o seu programa, parta de um cadastro e uma lista de notas de ginastas e teste as diversas funções, inclusive exercitando o resultado obtido (usando `it`) durante a execução destas funções em invocações posteriores de funções. Por exemplo, você pode testar uma sequência como:

```
adicionaGinasta cadastro (210, “Anne Frank”, “POL”)
adicionaGianasta it (300, “Elza Ricco”, “CUB”)
removeGinasta it 150
```